

Assignment 1 - Enter The Virtual World

Contribution to lab grade: 10%
Moodle Submission Deadline: 23:55, 07.11.2024
Presentation of your solution: 08.11.2024

In this assignment you will set up a Unity 3D project which you will continue to work on over the course of the semester. This includes importing dependency packages and an asset package that provides you with the basic functionality and a Unity scene. You will also set up a user avatar that tracks the position of your head-mounted display (HMD) and controllers. Finally you will learn how to use the ParrelSync plugin to test multi-user applications on a single computer, by joining a shared virtual scene with a VR and a Desktop user.

Follow the instructions for submitting and presenting assignments outlined in the “Lab Class and Assignment Instructions” document provided on Moodle.

.gitignore for Unity projects

We encourage you to use git as a version control system for your lab assignments (<https://git-scm.com/downloads>). The download archive for this assignment contains a .gitignore-file that you may use to ignore all temporary files generated by Unity.

Unity Project Setup

Exercise 1.1 (3 points)

To complete this task, set up a Unity project for VR and, in play mode, run the project on your HMD via Quest Link.

Create a new Unity project based on the “3D (Built-In Render Pipeline)” template.

Select Unity Editor Version 2022.3.50f1 when creating the new project.

If you are not familiar with Unity Editor, you can look up the [User Manual](#). You can get an overview of Editor’s interface and its different windows [on this page](#). Use [this page](#) as a starting point to get familiar with core concepts like Scenes, GameObjects and Prefabs.

Follow these steps in the Unity Editor to complete the project setup:

- ▶ Install the XR Plugin Management package using the [Package Manager](#)
- ▶ Enable the Oculus provider plug-in for Windows and Android platforms under **Edit > Project Settings > XR Plug-in Management**

Import the following packages using the [Package Manager](#):

- ▶ Authentication
- ▶ Lobby
- ▶ Relay
- ▶ Netcode for GameObjects
- ▶ XR Interaction Toolkit

To be able to use the multi-user functions in the project, it must be linked to a Unity Cloud Project. To link the project, navigate to **Edit > Project Settings > Services** in the menu. Select **VR Lab Class 2024** from the organization dropdown. In the second step, select **Use existing project**. From the Project dropdown, select **VR-Lab-Class-2024**. Click on **Link Unity project to cloud project**.

This step only works if you do it from a Lab computer that is logged into Unity with the VRSYS account! To set it up from a private computer, a Unity Developer account with a created organization and project is required.

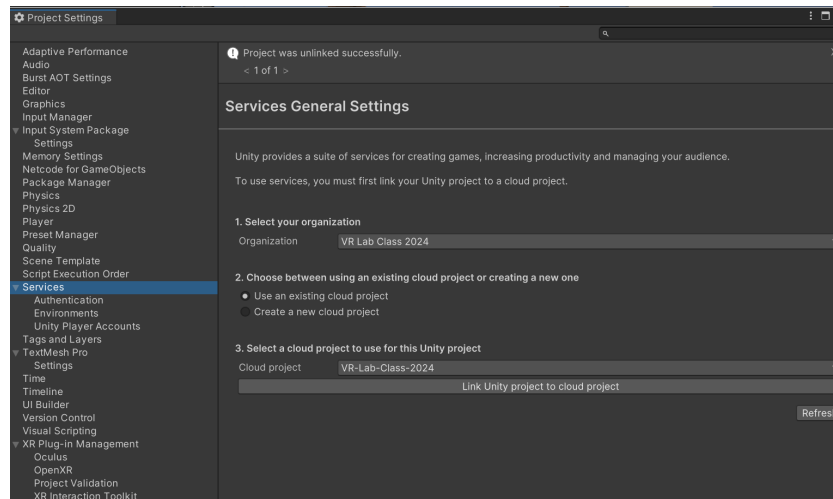


Figure 1: A snapshot of the correctly configured service settings.

Download the `vr-lab-2024-assignment-1.unitypackage` file from Moodle and [import the asset package](#) into your Unity project.

Now you can open the scene in `Assets/VR Course Assignment/Scenes/VRCourseAssignments.scene`.

In the menu under `File > Build Settings`, add the open scene to the build.

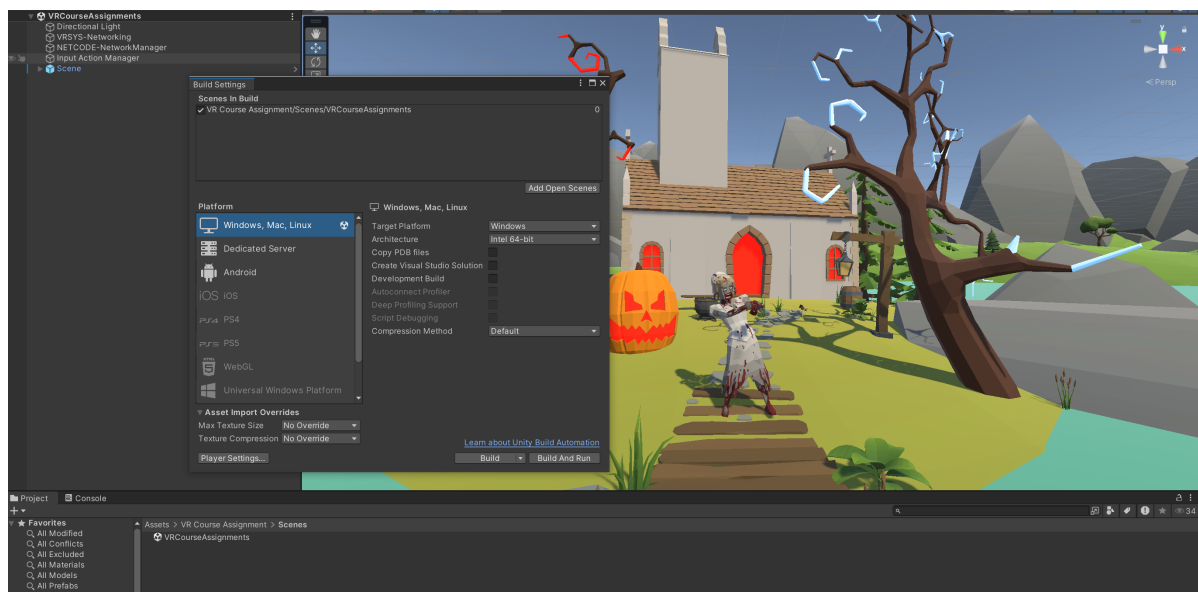


Figure 2: A snapshot of the correctly configured build settings.

You must complete the setup of your Quest and companion Windows programs (see Assignment Zero) to be able use the Unity Editor's play mode to start and run the project on your Quest.

Connect the Quest to the PC, allow USB-Debugging and start Quest Link. In build settings, you can check under android platform > run device > whether the Quest has been detected.

(Possible troubleshooting steps: Was Meta Quest Link started? Is Quest connected and active? Debugging allowed? Restart editor.)

Start the play-mode and check that the application runs on your headset.

Your Virtual Avatar

As you have now noticed, you will receive an image from the application on your headset, but it will not react to your head or hand movements. This is because the user prefab in Unity is not yet receiving any tracking data. Follow the next steps to familiarize yourself with setting up and using tracking from the headset and controllers in Unity.

Exercise 1.2 (2 point)

The entry point for tracking in Unity is the **XR Origin** component (= script added to a GameObject). First read up on the [XR Origin](#) and understand what this component is used for.

Now go to the Assignment Scene and create the XR Origin GameObject by right-clicking in the hierarchy **XR > XR Origin (VR)**, which comes with XR Interaction Toolkit package.

Then open the VR User Prefab under **Assets/VR Course Assignment/Assignment Materials/User Prefabs/HMDUser-VRCourse.prefab**.

Now add an XR Origin component to the root node of the User Prefab and configure it similar to the XR Origin example in your scene. Select Floor as Tracking Origin Mode.

Exercise 1.3 (3 point)

To actually apply the tracking to the user prefab, we can utilize the **Tracked Pose Driver (Input System)** component. This uses so-called Input Actions to read the tracking information from the headset and apply it to the corresponding node. First, familiarize yourself with the concept of [Input Actions](#). Under **Assets/VR Course Assignment/Input Actions**, you will already find an Input Actions Set containing preconfigured actions for tracking the head and hands.

Next, add Tracked Pose Driver (Input System) components to the corresponding nodes in the VR User Prefab and configure them with the Input Actions from the preconfigured

Input Action Set. Refer to the XR Origin example in the scene to identify the correct nodes.

In the XR Origin example, XR Controller components are used to read out the tracking instead of Tracked Pose Driver.

Next, remove the example XR Origin from the scene. Hit play. You should now be able to look around and move your hands.

Multi-User Testing & Networking Basics

VR is more fun with a friend! You already installed **Lobby and Relay**, dependencies of the scripts we provided you with. They are used to establish a lobby that users can connect to, creating a multi-player application. Unfortunately, you can only start the application for one user at a time in a Unity Editor. We use the ParrelSync plugin to get around this problem and thus be able to test with several users.

Exercise 1.4 (3 points)

Follow the installation instructions for [ParrelSync](#) on GitHub - this allows you to create a clone of your project for local multi-user testing. After the installation create a clone of your project and open it aswell.

In your original project select the **VRSYS-Networking** node and double click on the field for **Lobby Settings**. Enter your group name as the lobby name. Next, open the VR User Prefab. On the root node, find the **NetworkUser** component and unfold the **Local Behaviours** list. Add the XR Origin and all Tracked Pose Drivers, that you attached in the previous tasks, to the list.

In your clone project navigate to **Edit > Project Settings > XR Plug-in Management** and uncheck **Initialize XR on Startup** for the Windows and Android platform. Secondly in your scene select the **VRSYS-Networking** node and double click on the field for **User Spawn Info**. Choose **Desktop** from the user role dropdown.

Start the app in your original project as a VR user and in your clone as a Desktop user. You should now be able to see each other. The Desktop user can be navigated using mouse and WASD.

Exercise 1.5 (4 points)

We must serialize data, e.g. position data, in order to be able to see each others movements and interactions in a shared virtual environment.

We will use the [Netcode for GameObjects](#) framework for this in the assignments. Using Netcode, to enable the serialization of position, rotation and scale changes, only two components are required - NetworkObject and NetworkTransform. These are already configured in the VR & Desktop User Prefab so that movements are serialized.

To get started with serialization with Netcode, read the documentation to answer the following questions:

- ▶ What is the component NetworkObject used for?
- ▶ What is the component NetworkTransform used for?
- ▶ What is the difference between a NetworkTransform and a ClientNetworkTransform?
- ▶ Why do we use ClientNetworkTransforms at the User Prefabs.

Note down your answers in a text file and add it to your submission.